

Caching (Redis cache-aside)

Status: Implemented locally (learning). **Off in prod** by default.

Feature Overview

BankOne uses **cache-aside** with Redis so hot **read** APIs can skip Postgres when a fresh enough copy already exists in Redis.

```
flowchart LR
    client[Client] --> svc[Service_read]
    svc --> redis{Redis_hit}
    redis -->|yes| ret[Return_cached]
    redis -->|miss| db[(Postgres_or_replica)]
    db --> put[Put_Redis_with_TTL]
    put --> ret
    write[Service_write] --> evict[Evict_related_keys]
    evict --> dbw[(Write_Postgres)]
```

- **Not** write-through: writes go to the DB first; then related cache keys are cleared.
- **Not** a full DB mirror: Redis only holds what was read recently (lazy fill on miss).
- **Local only:** `app.cache.redis-enabled=true` by default locally; `application-prod.properties` sets it to `false`.

Same Redis host as login rate-limiting (`spring.data.redis.host` / `REDIS_HOST`). Different key namespaces:

Concern	Keys / store
Entity / API cache	<code>bankone:<cacheName>:...</code> (Spring Cache)
Rate-limit buckets	Bucket4j keys (separate from entity cache)

How it works (request path)

1. Staff/portal calls a read API (e.g. `GET /customers/{id}`, search, roles, dashboard).
2. Spring `@Cacheable` runs **before** the service method body.
3. **Hit:** value deserialized from Redis → returned (no repository call for that method).
4. **Miss:** method runs → loads from Postgres (often via `@Transactional(readOnly=true)` / read replica when enabled) → result stored in Redis with the region TTL → returned.
5. On create/update/deposit/etc., `@CacheEvict` clears related regions so the next read refills from DB.

Auth (`CustomUserDetailsService` / JWT parse) is **not** cached.

Configuration

Property	Default (local)	Notes
<code>app.cache.redis-enabled</code>	<code>true</code>	<code>APP_CACHE_REDIS</code> ; prod <code>false</code>
<code>app.cache.ttl-seconds</code>	<code>300</code>	Fallback TTL; named caches override
<code>spring.data.redis.host</code>	<code>localhost</code>	Shared with rate limit
<code>spring.data.redis.port</code>	<code>6379</code>	Docker Compose Redis service
<code>`spring.data.redis.repositories.e</code>	nabled <code>false`</code>	Avoid treating JPA repos as Redis repos

Code: `com.bankone.cache` —

- `RedisCacheConfig` — `@EnableCaching` + `RedisCacheManager` when enabled
- `CacheNames` — region name constants
- `CacheKeys` — stable hashes for search/page keys
- `CacheProperties` — binds `app.cache.*`

Cache regions and TTLs

Region (<code>CacheNames</code>)	Typical keys	TTL
<code>customers</code>	<code>id:{id}</code> , <code>all</code> , <code>search:{hash}</code>	5 min
<code>accounts</code>	<code>id:{id}</code> , <code>customer:{id}</code> , <code>search:{hash}</code>	2 min
<code>policies</code>	<code>active:{type}:{ccy}</code> , <code>all</code>	30 min
<code>roles</code>	<code>all</code> , <code>id:{id}</code>	30 min
<code>users</code>	<code>employees:{hash}</code>	5 min
<code>transactions</code>	<code>account:... </code> , <code>staff:... </code>	60 s
<code>beneficiaries</code>	<code>user:{customerId}</code>	5 min
<code>dashboard</code>	<code>summary</code>	60 s
<code>transfers</code>	<code>pending</code> , <code>history:{user}</code> , <code>audit</code>	30 s
<code>reports</code>	<code>trends / mix / approvals</code>	60 s

Redis key example: `bankone:customers::id:20120`

(`bankone:` prefix + cache name + `::` + SpEL key).

Portal account/tx reads reuse **accounts** / **transactions** caches via `AccountService` / `TransactionService` (no separate portal store).

Eviction rules

- Customer create/update/delete → clear `customers` (+ often `accounts` / `dashboard` / `users` when onboarding creates portal login or accounts).
- Account open / status / deposit / withdraw / transfer → clear `accounts`, `transactions`, `dashboard`, `reports` (and `transfers` on transfer).
- Policy / role / employee mutate → clear that region (`allEntries`).
- Beneficiary add/remove → clear `beneficiaries`.
- Transfer approve/reject / portal transfer create → clear `transfers` (and related money caches when ledger moves).

For local lab size, eviction often uses `allEntries=true` on the region so search pages cannot go stale.

Serialization

Values use **JDK serialization** (`RedisSerializer.java()`), not Jackson default-typing.

Why: Jackson root `List` / `PageImpl` failed to deserialize after search (UI errors on second search / roles). JDK serialization round-trips `Page` and `List` reliably when cached types implement `Serializable` (entities/DTOs/records used in cache do).

After changing serializer format, **flush** old keys:

```
docker exec bankone-redis-1 redis-cli EVAL \
  "local k=redis.call('keys',ARGV[1]); for i=1,#k do redis.call('del',k[i]) end; return #k" \
  0 'bankone:*
```

What is intentionally not cached

- Login / JWT validation / `CustomUserDetailsService`
- Shard-lab vertical slice
- Rate-limit buckets (Bucket4j's own Redis keys)
- Live balances **without** eviction — money writes clear account/tx caches so the next read is fresh from DB

How to verify (local)

Prerequisites: Redis up (`docker compose Redis` on `:6379`), Liberty with `APP_CACHE_REDIS=true` (default local).

```
# 1) Clear keys
docker exec bankone-redis-1 redis-cli --scan --pattern 'bankone:*'

# 2) Login + GET once (miss → key appears)
# GET /customers/{id} or search /customers?search=...

# 3) Same GET again (hit → same key; no new SELECT for that method)

# 4) Update customer / deposit → related keys gone
```

Or watch Hibernate SQL: first call shows `SELECT`; second call for the same cached method should not.

E2E / manual: Management UI search customers twice; second load should succeed (no 500 from Redis deserialize).

Interaction with read replica

When `app.replica.enabled=true`, `@Transactional(readOnly=true)` reads may hit `bankone_read`. Cache sits **in front** of that path:

- Miss → may read replica → store in Redis
- Hit → Redis only (no replica round-trip)
- After write on primary, eviction ensures next miss can see new data (replica lag is a separate concern — use Sync replica / wait for scheduler if testing replica lag)

Future Modification Guide

Change	Touch
New cacheable read	Add <code>@Cacheable</code> + key; ensure return type is <code>Serializable</code> ; pick region / TTL in <code>RedisCacheConfig</code>
New write that changes lists	Add <code>@CacheEvict</code> / <code>@Caching</code> on the writer
Change TTL	<code>RedisCacheConfig</code> per-cache map
Enable in prod	Only with Redis + eviction review; set <code>app.cache.redis-enabled=true</code> carefully
Drop caching	<code>APP_CACHE_REDIS=false</code> — annotations become no-ops (no <code>CacheManager</code>)

Call hierarchy (read hit/miss)

```
Controller → ServiceImpl.@Cacheable
              → CacheAspect → RedisCache.lookup
                  hit → return cached
                  miss → ServiceImpl body → Repository → put Redis
```

Call hierarchy (write)

```
Controller → ServiceImpl.@CacheEvict → Repository save
              → Redis region cleared
```

Related: [ARCHITECTURE.md](#), [TECH_STACK.md](#), [TECH_LEARNING_PLAN.md](#).